

Computational approaches in Bayesian inference

- Recall that given data $\vec{y} \in \mathbb{R}^N$, inputs $\vec{x} \in \mathbb{R}^{n \times d}$, + model $f(\underline{x}; \vec{\theta})$, $\vec{\theta} \in \mathbb{R}^p$ with

$$y_i = f(\underline{x}_i; \vec{\theta}) + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

- Then given some prior $P(\vec{\theta})$ on parameters

$$\Rightarrow P(\vec{\theta} | \vec{y}) = \frac{P(\vec{y} | \vec{\theta}) P(\vec{\theta})}{\int_{\mathbb{R}^p} P(\vec{y} | \vec{\theta}) P(\vec{\theta}) d\vec{\theta}}$$

- In practice, we cannot get a closed-form solution for evidence in the denominator or the entire posterior,

- We would like to draw samples from $P(\vec{\theta} | \vec{y})$ + determine if they are "likely" or not

□ So given $P(\vec{y} | \vec{\theta})$ (which compares $f(\underline{x}_i; \vec{\theta})$ to y_i) + $P(\vec{\theta})$

Markov Chain Monte Carlo

→ we can't calculate $P(\vec{\theta} | \vec{y})$ analytically

- In general, we cannot solve or write down $P(\vec{\theta}|\vec{y})$ analytically
 so we draw samples of $\vec{\theta}$, & use $P(\vec{\theta})$ & $P(\vec{y}|\vec{\theta})$ to
 accept or reject that value.

- A Markov chain is a random process where the next value in
 that process only depends on its current state

- I.e., the next step depends on the present, not the past
- It is memory-less

- Monte Carlo is randomized selections of points based on a distribution

- E.g., if $m \sim N(0, \sigma^2)$ we can draw samples from this in
 a "Monte Carlo" style.

- As we draw random samples, we slowly approximate the true distribution,

- Markov Chain Monte Carlo (MCMC) draws random, Markov chain samples, & then
 slowly approximates $P(\vec{\theta}|\vec{y})$.

- Basic outline



